

COMP101  
Introduction to Programming  
2025-26

Lecture-20  
Parameter Return

# Calling functions – no value returned

- Call with no argument (in the caller brackets)
- and no parameter (in the function brackets) and no return
  - The function works with data already inside the function
  - The data is 'local to the function' (is 'bounded by the scope of the function')
  - The data can't be used outside the function
- Call with an argument (in the caller brackets)
- and a parameter (in the function brackets) and no return
  - A copy of the data is passed to the function
  - The data is 'local to the function' (is 'bounded by the scope of the function')
  - Data can't be used outside the function

# Value not returned to caller

#function to count characters

```
def num_length (number_str):  
    length_of_num = (len(number_str))  
    print ("Your number has",length_of_num,"digits in it")  
    return
```

```
def main():  
    num = int (input("Enter an integer: "))  
    num = str (num) #cast to string for any digits  
    num_length (num) #caller
```

```
main()
```

`length_of_num` is a 'local variable' – it is declared within the function and is used only in the function;  
`number_str` which has been received as a parameter is also a 'local variable' – it is used only in the function

'local variables' only exist within the 'scope' of the function and cannot be used outside the function.

`return` (can use `return()`) shows function has ended, but it does not do anything – there is 'no return of value'. It defaults as 'None'. Good practice to use it.

# Calling functions - value returned

- Calling a function with an argument (in the caller brackets) and a parameter (in the function brackets)
- The parameter is local to the function and is used as a 'local var' within the function.
- The 'local var' can be changed by the function and the changed 'local var' can be returned to the caller for rest of the program to use
- We extend the 'return' statement to allow a value in a function to be returned out of the function to the caller
- The 'return' statement is usually the last statement in a function

# Value returned to caller

Outside the function:

1. Call the function using a variable as an argument
2. The variable will capture the value returned from the function

Inside the function:

3. Use the parameter name to perform any operations
4. Use 'return ()' with the parameter name in brackets to pass the value out of the function back to the caller

# Returning a value

```
def squared(value):  
    value = value * value #local variable value=25  
    return (value)         #25 is returned to the caller
```

```
def main():  
    num = 5  
    #need a var to call the function. This one is 'num_squared'  
    num_squared = squared (num)  
    print (num_squared) #will print 25
```

```
main()
```

In main():

variable `num_squared` calls function `squared` with argument=5

In squared(value):

`squared` accepts argument `num` as alias parameter `value`

`value` is now a local variable – only exists in the function

`return` uses local variable `value` in brackets as an argument to give the answer back to the calling variable `num_squared`.

`value` is the 'return value'

In def main():

`num_squared` receives the returned value of 25

Execution continues at the line after the caller, so we print 25

We now have var `num` = 5 and var `num_squared` =25

When we call a function and give it an argument, we actually **pass a copy of the argument**:

Hence `num` will still be 5 after the function has finished and we have another var `num_squared` with value 25

# Returning a value

```
def myGrades(a,b,c,d):  
    print ("Calculating average of", a,b,c,d)  
    average = (a+b+c+d)/4  
    return (average)
```

```
def main():  
    ca01 = 9  
    ca02 = 20  
    ca03 = 34  
    ca04 = 45
```

```
#finalScore (as caller) will receive the answer from the function  
finalScore = myGrades (ca01, ca02, ca03, ca04)  
print ("Average of scores =", finalScore)
```

```
main()
```

The function uses 'return()' to return an 'argument' to the caller.

The argument here is a local variable ('average') that has been assigned the calculation.

If we don't give 'return' any arguments, it will return the value 'None'.

In `def main():`

We have five vars - `ca01`, `ca02`, `ca03`, `ca04` and `finalScore`

Use var `finalScore` to call the function and pass four arguments:

argument `Ca01` passes to parameter `a`

argument `ca02` passes to parameter `b` etc

In `def(myGrades())` we use local var `average` to calculate average

Return `average` to the caller `finalScore`

In `main` there are still five vars:

`ca01`, `ca02`, `ca03`, `ca04` with the original values (9,20,34,45) and `finalScore` with a new value of the average created by the function

# Passing arguments - general

Information only

- Call-by-Value

- Copy of argument passed to a function
- Copy can be changed inside the function
- Original value (outside the function) is unchanged

- Call-by-Reference ( 'Call-by-Object', 'Call-by-Object-Reference', 'Call-by-Sharing' )

- Original argument is passed to a function
  - Function is allowed access to argument's memory location
- Original can be changed inside the function
- Original value (outside the function) is changed



# Python - how it passes arguments

Information only

- Python passes by 'assignment'
- It uses the data type to determine if Python:
  - calls-by-value (passes a copy) or
  - passes-by-reference (passes the original)
- Immutable objects are passed-by-value
  - numbers, strings, tuples
- Mutable objects are passed-by-reference
  - lists, dictionaries

# Declaring functions – Ass-05

def prime():

loop

i/p single int – process prime y/n – process even/odd – o/p

no return

def ice-in():

i/p ints for floe-01, i/p ints for floe-02

return (ints)

def ice-calc():

use the return from ice-in to calc volumes for both floes and calc average vol

return (floe-01, floe-02, average)

def ice-out():

use the return from ice-calc to o/p vol floe-01, vol floe-02, average

no return

Functions can be declared in any order and before def main().

Ideally a function does one job and does it well – although some extra 'support code' is okay

# Declaring main – order of calls

def main():

    prime()

    ice-in()

    ice-calc()

    ice-out()

main()

Functions can be declared in any order but must be before def main();

Call functions in the order you want them to execute;  
prime is a standalone function – it could be called after the ice-floe set as it does not interact with any other function – it has no return

Order of call for the ice-floe is critical as it needs three calls to complete its execution;  
ice-in must be called before ice-calc;  
ice-calc must be called before ice-out

def main() controls execution of the functions using 'calls'.  
Inside def main() can be loop control but not code such as i/p, o/p etc – these belong in the functions that are called

# Call function having no return

```
def funct-01(num):  
    process num  
    no return
```

```
def main():  
    input(num)  
    funct-01(num)  
    funct-02()  
    funct-03()  
    funct-04()
```

```
main()
```

Keep the integrity of functions  
main() should handle calls to functions - only  
Avoid other processing except loop controls in it.

i/p should be done in funct-01 not in def main()

# Call a function having no return of values

```
def funct-01():
```

```
    i/p num
```

```
    process num
```

```
    o/p answer
```

```
    no return
```

```
def main():
```

```
    funct-01()
```

```
    funct-02()
```

```
    funct-03()
```

```
    funct-04()
```

```
main()
```

main() is not compromised and all data handling is in the function;

# Call function using loop

```
def funct-01():  
    i/p num  
    process num  
    o/p answer  
    no return
```

```
def main():  
    loop  
        funct-01()  
    until stop
```

```
main()
```

main() is not compromised and all data handling is in the function;

Main here controls the iteration of the loop.

**This is okay but study the function - it does all of its own processing without affecting any other functions.**

**Function integrity is best served with the loop inside the function.**

**If the function was needed by another program the loop control stays with the function.**

# Call function – loop inside function

```
def funct-01():  
    loop  
    i/p num  
    process num  
    o/p answer  
    until stop  
    no return
```

```
def main():  
    funct-01()
```

```
main()
```

main() is not compromised and all data handling is in the function

# function with return

```
def funct-02():  
    i/p int  
    return (int)
```

```
def main():  
    funct-02()
```

```
main()
```

Here the call here is to funct-02:  
But it won't work.

funct-02 returns a value which is allowed to leave  
the function to be used by another function.

The call needs amending to handle the data-  
passing mechanism



# Call managing return values

```
def funct-02():  
    i/p int  
    return (int)
```

```
def main():  
    varName = funct-02()
```

```
main()
```

The amendments are in colour text

We need a var to hold the i/p value from funct-02.

This var will be used in other functions.

funct-02 () is still declared with empty brackets – we are not passing any data into it – the data is generated within the function itself.

What is i/p is to be returned to the caller for further processing by another function;

# Call mangling arguments and parameters and return

```
def funct-03(varName):
```

```
    use the return from funct-02 and square it  
    return (square)
```

```
def main():
```

```
    varName = funct-02()
```

```
    varSquared = funct-03 (varName)
```

```
main()
```

def funct-03 is the next call

We can't declare this with empty brackets – it needs the var created in funct-02 to be passed to it;

funct-03 returns the answer. Having a return of values means it needs calling with vars to be able to hold the return values.

# Call managing arguments and parameters and no return

```
def funct-04(varSquared from funct-03):
```

```
    print (var)
```

```
    no return
```

```
def main():
```

```
    funct-04 (varSquared)
```

```
main()
```

def funct-04 is the next (and final) call

We can't declare this with empty brackets – it needs the var returned in funct-03

It has no return of values so we don't need vars to be part of the call

# Summary – basic structure

def funct-01(): **#no return**

def funct-02(): **#return – needs var call**

def funct-03(): **#return – needs var call with argument and params**

def funct-04(): **#no return with argument and params**

def main():

    funct-01()

    varName = funct-02()

    varSquared = funct-03()

    funct-04()

main()

## **ASS-05:**

funct-01 - place the loop in the function to improve the modularity – all operation is now in one function;

Funct-02/03/04 should be loop-controlled in def main();  
The suite of functions, if needed to be imported into another program, 'belong' together in order to work.

# Summary – calls structure and funct control

```
def funct-01():          #accept no parameters, return no arguments
def funct-02():          #accept no parameters, return arguments
def funct-03(parameter): #accept parameters, return arguments
def funct-04(parameter): #accept parameters, return no arguments
```

Execution requires funct-02,  
funct-03 and funct-04 to execute  
as one function suite

```
def main():
    funct-01()
    loopPrompt
    loop
        variable = funct-02()
        variable = funct-03(argument)
        funct-04(argument)
    loopPrompt
    #if more functs don't indent them
main()
```

We can't always maintain true function integrity – sometimes it isn't practical. To loop funct-02/funct-03/funct-04 suite we must have i/o etc in def main() as well as the calls.

def main() controls the calls to the functions. Part of control here needs loop control, hence having i/o is expected as it is part of the control needed.

functs that return values need to be called using  
varName = functName()

The number of vars needed in the call match the number of values to be returned. Note positional behaviour.